Linux eBPF: Running Code Inside OS Kernel Without Reboot

How eBPF lets engineers execute sandboxed bytecode at 100Gbps network line-rate without crashing the host.

⚡ 1. The Kernel Boundary Problem

Traditional Linux tools run in User Space. Every packet inspection requires a heavy context switch and memory copy from Kernel Space.

eBPF Superpower: Hooks directly into kernel tracepoints, kprobes, and XDP (eXpress Data Path) network driver buffers.

- **Zero Context Switch:** Runs directly inside kernel execution path.
- **Zero Reboot:** Load and unload bytecode on a live production server safely.

🛡️ 2. The In-Kernel Bytecode Verifier

Before any eBPF program runs, the Linux kernel verifier strictly checks all execution paths:

- **No Infinite Loops:** Guarantees program will always terminate.
- **Safe Memory Access:** Pointer arithmetic is checked against buffer boundaries.
- **Privilege Guard:** Prevents unauthorized memory reads; zero risk of kernel panic!

🚀 3. JIT Compilation to Native Machine Code

Once verified, the kernel JIT (Just-In-Time) compiler compiles eBPF bytecode directly into raw x86/ARM machine instructions:

```
Bytecode (LLVM Clang) -> Kernel Verifier -> JIT Compiler -> Native CPU Execution
```

Executes at the physical hardware clock speed with sub-microsecond overhead.

💻 4. 2-Minute Hello World (Python BCC)

Monitor all process executions on your Linux system in real time:

```
# 1. Install BCC tools
sudo apt install -y bpfcc-tools linux-headers-$(uname -r)

# 2. Run live trace of all system commands
sudo execsnoop-bpfcc

# Output: Shows PID, command name, and arguments instantly!
```